SECURE CODING REVIEW

CodeAlpha Cyber Security Internship — Task 3

Application Audited: Simple Python Flask Login Application

Language: Python 3 + Flask

Prepared by: **Momina Bibi**
CodeAlpha Cyber Security Intern

1. Introduction

This report presents a secure code review of a simple Python Flask-based login application. The purpose of this review is to identify common security vulnerabilities, explain their impact, and provide clear remediation steps following secure coding best practices.

The review was performed using a combination of manual code inspection and knowledge of common vulnerability patterns (OWASP Top 10). The findings are suitable for educational and internship demonstration purposes.

2. Scope of Review

- Programming Language: Python 3
- Framework: Flask (web application)
- Focus Areas: Authentication, Input Handling, Data Storage, Session Management
- Method: Manual static code review + security best practice checklist

3. Sample Vulnerable Application Code

The following simplified Flask application was used for the audit. It contains several intentional security weaknesses commonly found in beginner-level projects.

```
from flask import Flask, request, render_template_string
import sqlite3

app = Flask(__name__)
app.secret_key = "mysecret123"          # Hardcoded secret

@app.route("/login", methods=["POST"])
def login():
    username = request.form["username"]
    password = request.form["password"]

    # Vulnerable SQL query (SQL Injection)
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username='" + username + \
            "' AND password='" + password + "'"
    cursor.execute(query)
    user = cursor.fetchone()

    if user:
        return "Welcome " + username     # Possible XSS
    else:
        return "Login Failed"

@app.route("/search")
def search():
    q = request.args.get("q", "")
    # Reflected XSS
    return "<h1>Results for: " + q + "</h1>"

if __name__ == "__main__":
    app.run(debug=True)                 # Debug mode in production
```

4. Security Findings

During the review, the following vulnerabilities were identified:

Finding 1: SQL Injection (Critical)

Location: login() function – SQL query construction

User input (username and password) is directly concatenated into the SQL query string. An attacker can inject malicious SQL (e.g. ' OR '1'='1) to bypass authentication or extract data.

Impact: Complete authentication bypass, data theft, database compromise.

OWASP Category: A03:2021 – Injection

Finding 2: Cross-Site Scripting – XSS (High)

Location: search() route and welcome message

User-controlled input is reflected directly into the HTML response without sanitization or encoding. An attacker can inject JavaScript payloads that execute in the victim's browser.

Impact: Session hijacking, credential theft, defacement.

OWASP Category: A03:2021 – Injection / A07:2021 – XSS

Finding 3: Hardcoded Secret Key (Medium)

Location: app.secret_key = "mysecret123"

The Flask secret key is hardcoded in the source code. If the code is leaked (GitHub, backup, etc.), attackers can forge session cookies and impersonate users.

Impact: Session forgery, account takeover.

Finding 4: Debug Mode Enabled (Medium)

Location: app.run(debug=True)

Running the application with debug=True in a production or publicly accessible environment exposes the interactive debugger, which can lead to remote code execution.

Impact: Remote code execution, full server compromise.

Finding 5: Plaintext Password Storage (High)

Location: Password comparison in SQL query

Passwords appear to be stored and compared in plaintext. If the database is compromised, all user credentials are immediately usable by attackers.

Impact: Credential theft, account takeover across services.

5. Summary of Vulnerabilities

ID	Vulnerability	Severity	OWASP
1	SQL Injection	Critical	A03:2021
2	Cross-Site Scripting (XSS)	High	A03/A07
3	Hardcoded Secret Key	Medium	A02:2021
4	Debug Mode Enabled	Medium	A05:2021
5	Plaintext Password Storage	High	A02:2021

6. Recommendations & Remediation Steps

6.1 Fix SQL Injection – Use Parameterized Queries

Never concatenate user input into SQL. Always use parameterized queries (prepared statements).

```
# SAFE version
cursor.execute(
    "SELECT * FROM users WHERE username=? AND password=?",
    (username, password)
)
```

6.2 Prevent XSS – Escape Output

Use Jinja2 auto-escaping (default in Flask render_template) or explicitly escape user data. Never build HTML by concatenating strings with user input.

```
# SAFE version
from flask import escape
return f"<h1>Results for: {escape(q)}</h1>"
# Or better: return render_template("search.html", query=q)
```

6.3 Store Secret Key Securely

Load the secret key from environment variables or a secrets manager. Never hardcode it.

```
import os
app.secret_key = os.environ.get("SECRET_KEY")
# Generate a strong key: python -c "import secrets; print(secrets.token_hex(32))"
```

6.4 Disable Debug Mode in Production

Set debug=False or control it via environment variable. Never deploy with debug=True.

6.5 Hash Passwords Properly

Use a strong password hashing algorithm such as bcrypt, scrypt, or Argon2. Never store or compare plaintext passwords.

```
from werkzeug.security import generate_password_hash, check_password_hash

# When registering:
hashed = generate_password_hash(password)

# When logging in:
if check_password_hash(stored_hash, password):
    # success
```

7. Secure Coding Best Practices

- Validate and sanitize all user inputs on both client and server side.
- Apply the principle of least privilege for database and file access.
- Keep frameworks, libraries, and dependencies up to date.
- Use HTTPS everywhere and set secure cookie flags (HttpOnly, Secure, SameSite).
- Implement proper error handling — do not leak stack traces to users.
- Perform regular code reviews and use static analysis tools (Bandit, Semgrep, SonarQube).
- Follow OWASP Top 10 and language-specific secure coding guidelines.

8. Recommended Tools for Future Reviews

- **Bandit** — Python security linter (static analysis)
- **Semgrep** — Fast static analysis for many languages
- **SonarQube / SonarCloud** — Comprehensive code quality & security
- **OWASP ZAP** — Dynamic application security testing
- **Manual Review** — Always combine tools with human inspection

9. Conclusion

The reviewed application contains several critical and high-severity vulnerabilities that could allow attackers to bypass authentication, steal sessions, or execute malicious scripts. All identified issues have clear and practical remediation steps. Implementing parameterized queries, output encoding, proper password hashing, and secure configuration would significantly improve the security posture of the application.

This exercise demonstrates the importance of secure coding practices from the earliest stages of development. Security should be built in, not added later.

Report prepared for CodeAlpha Cyber Security Internship – Task 3

Author: Momina Bibi